Technische Universität Braunschweig

Chair for Chip Design for Embedded Computing

HWSW-Systeme

Übung 04: VHDL - bedingte Zuweisungen

M.Sc. Eike Trumann, M.Sc. Jasper Homann,

Prof. Dr.-Ing. Guillermo Payá Vayá

14. November 2025

1 Hardwarebeschreibung und Implementierung

Die folgende Beschreibung implementiert eine ALU (Arithmetic-Logic-Unit) mit mehreren Funktionen.

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity alu is
port(
        clk          : in  std_ulogic;
        rst          : in  std_ulogic;
        opcode       : in  std_ulogic_vector(1 downto 0);
        clear_result : in  std_ulogic;
        a            : in  std_ulogic_vector(31 downto 0);
        b            : in  std_ulogic_vector(31 downto 0);
        result       : out std_ulogic_vector(31 downto 0)
);
end entity alu;
```

Kurzfragen

Wie viele verschiedene Operationen könnte dieses Modul ausführen?

Implementierung 1

```
architecture RTL of alu is
begin
 process(a, b, opcode)
 begin
        result <= a;
        case (opcode) is
                when "00" =>
                        result <= a and b;
                when "01" =>
                        result <= a or b;
                when "10" =>
                        result <= not a;
                when "11" =>
                        result <= a;
                when others => null;
        end case;
 end process;
end architecture RTL;
```

Wie sieht diese Implementierung in Hardware aus?

Implementierung 2

```
architecture RTL of alu is
begin
 process(a, b, opcode)
 begin
        result <= a;
        if (opcode = "00") then
                result <= a and b;
        elsif (opcode = "01") then
                result <= a or b;
        elsif (opcode = "10") then
                result <= not a;
        end if;
 end process;
end architecture RTL;
```

Wie sieht diese Implementierung in Hardware aus?

Implementierung 3

```
architecture RTL of alu is
begin
 result <= a and b when (opcode = "00") else
           a or b  when (opcode = "01") else
           not a   when (opcode = "10") else
           a;
end architecture RTL;
```

Wie sieht diese Implementierung in Hardware aus?

Implementierung 4

```
architecture RTL of alu is
begin
 process(a, b, opcode, clear_result)
 begin
        result <= a;
        if (clear_result = '1') then
                result <= (others => '0');
        elsif (opcode = "00") then
                result <= a and b;
        elsif (opcode = "01") then
                result <= a or b;
        elsif (opcode = "10") then
                result <= not a;
        elsif (opcode = "11") then
                result <= a;
        end if;
 end process;
end architecture RTL;
```

Wie sieht diese Implementierung in Hardware aus?

2 Fehlersuche

Finden Sie Fehler in folgender Hardwarebeschreibung?

```
process(opcode, input)
begin
        case (input(02 downto 00)) is
                when "000" | "001" | "010" =>
                        output(3 downto 0) <= "1111";
                when "011" | "100" | "101" =>
                        output(3 downto 0) <= a(3 downto 0)
                when "110" | "111" =>
                        output(3 downto 0) <= "0000";
        end case;
end process;
```

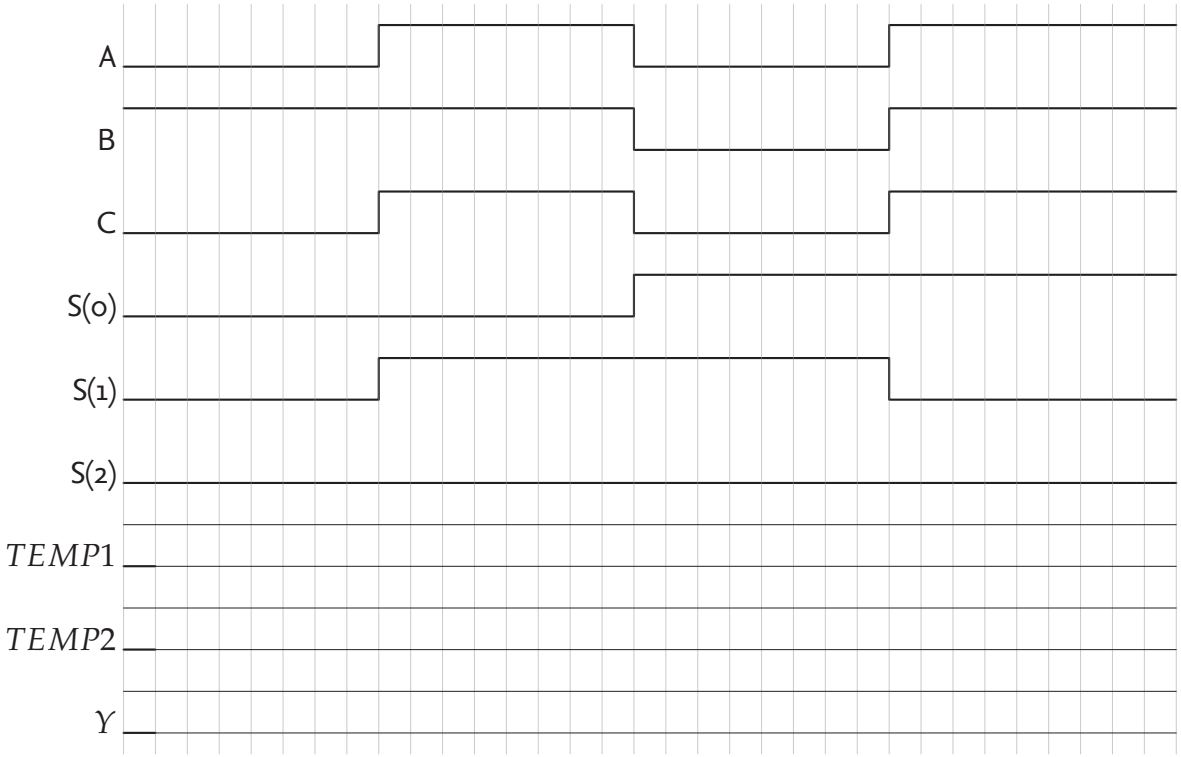
3 Kurzfragen

Wozu dienen sensitivity-Listen in der Simulation?

4 Timing Diagramm

Füllen Sie das Timing Diagramm für die folgende Schaltung aus:

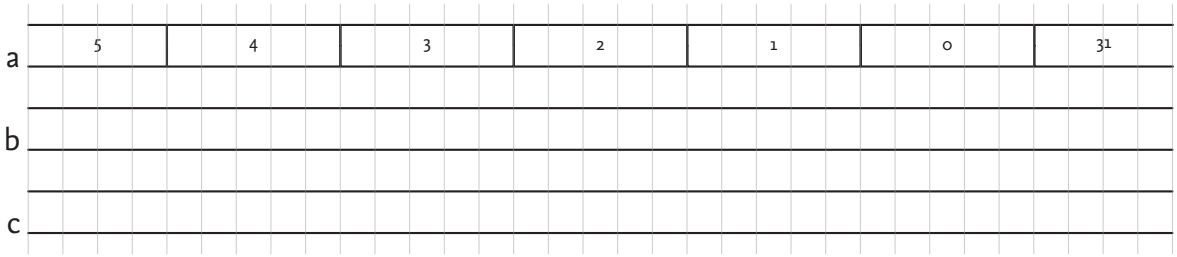
```
entity BEISPIEL is
 port (A, B, C : in  std_ulogic;                    -- Dateneingang
       S       : in  std_ulogic_vector(2 downto 0); -- Selektionseingang
       Y       : out std_ulogic);                   -- Ausgang
end entity;

architecture RTL of BEISPIEL is
    signal TEMP1, TEMP2 : std_ulogic;
begin
  MUX1: process (A, TEMP1, S(0))
  begin
      TEMP2 <= (A and not S(0)) or (TEMP1 and S(0));
  end process MUX1;

  MUX2: process (B, C, S(1))
  begin
      TEMP1 <= (B and not S(1)) or (C and S(1));
  end process MUX2;

  Y <= TEMP2 xor S(2);
end architecture RTL;
```



Timing-Diagramm 2

Füllen Sie das Timing Diagramm für die folgende Schaltung aus:

```
process(a, c)
begin
        c <= 2 + a;
        b <= 1;
        if (c = 2) then
                b <= 4;
        end if;
end process;
```



5 VHDL

Wie sieht die Schaltung zu folgender Hardwarebeschreibung aus?

```
entity ASDF is
generic (BITS   : integer := 4);
port (D      : in std_ulogic_vector(BITS-1 downto 0);
      JA     : out std_ulogic);
end ASDF;
architecture RTL of ASDF is
begin
        process (D)
                variable result : std_ulogic;
        begin
                result := '0';
                for I in BITS-1 downto 0 loop
                        result := D(I) or result;
                end loop;
                JA <= result;
        end process;
end architecture RTL;
```

Entwickeln Sie eine Architektur, die die Anzahl der Nullen in einem Vektor zählt (Count Zeros). Beispiel: Der Eingang $data = \text{"0001011"}$ hat 4 Nullen, folglich soll das Ergebnis den Wert 4 haben.

```
        entity CZ is
                generic(
                        DATA_WIDTH  : natural := 32;
                        COUNT_WIDTH : natural := 6   );
                port (
                        data  : in  std_ulogic_vector( DATA_WIDTH - 1 downto 0);
                        zeros : out std_ulogic_vector(COUNT_WIDTH - 1 downto 0)   );
        end entity;

        architecture RTL of CZ is
          signal count : unsigned(COUNT_WIDTH - 1 downto 0);
        begin
```

```
        -- EUER CODE VON HIER

        -- BIS HIER
        zeros <= std_ulogic_vector(count);
end architecture RTL;
```

Kurzfragen

Wie sieht die Hardware aus?
Wo verläuft der kritische Pfad dieser Architektur?
Wie lang ist der kritische Pfad, wenn **DATA_WIDTH** auf 64 gesetzt wird?

VHDL Implementierung

Wie kann man die Architektur besser/schneller aufbauen?
Entwickeln Sie eine effizientere Hardwarearchitektur für $DATA_WIDTH = 4$.

```
-- EUER CODE VON HIER

-- BIS HIER
```